

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao
Princeton University

Dian Yu
Google DeepMind

Jeffrey Zhao
Google DeepMind

Izhak Shafran
Google DeepMind

Thomas L. Griffiths
Princeton University

Yuan Cao
Google DeepMind

Karthik Narasimhan
Princeton University

Abstract

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference. This means they can fall short in tasks that require exploration, strategic lookahead, or where initial decisions play a pivotal role. To surmount these challenges, we introduce a new framework for language model inference, “Tree of Thoughts” (ToT), which generalizes over the popular “Chain of Thought” approach to prompting language models, and enables exploration over coherent units of text (“thoughts”) that serve as intermediate steps toward problem solving. ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices. Our experiments show that ToT significantly enhances language models’ problem-solving abilities on three novel tasks requiring non-trivial planning or search: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%. Code repo with all prompts: <https://github.com/princeton-nlp/tree-of-thought-llm>.

1 Introduction

Originally designed to generate text, scaled-up versions of language models (LMs) such as GPT [25, 26, 1, 23] and PaLM [5] have been shown to be increasingly capable of performing an ever wider range of tasks requiring mathematical, symbolic, commonsense, and knowledge reasoning. It is perhaps surprising that underlying all this progress is still the original autoregressive mechanism for generating text, which makes token-level decisions one by one and in a left-to-right fashion. Is such a simple mechanism sufficient for a LM to be built toward a general problem solver? If not, what problems would challenge the current paradigm, and what should be alternative mechanisms?

The literature on human cognition provides some clues to answer these questions. Research on “dual process” models suggests that people have two modes in which they engage with decisions – a fast, automatic, unconscious mode (“System 1”) and a slow, deliberate, conscious mode (“System 2”) [30, 31, 16, 15]. These two modes have previously been connected to a variety of mathematical models used in machine learning. For example, research on reinforcement learning in humans and other animals has explored the circumstances under which they engage in associative “model free” learning or more deliberative “model based” planning [7]. The simple associative token-level choices of LMs are also reminiscent of “System 1”, and thus might benefit from augmentation by a more deliberate “System 2” planning process that (1) maintains and explores diverse alternatives for current

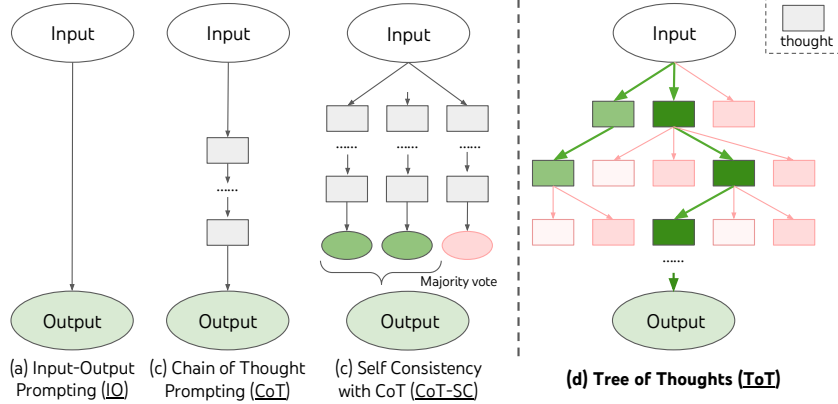


Figure 1: Schematic illustrating various approaches to problem solving with LLMs. Each rectangle box represents a *thought*, which is a coherent language sequence that serves as an intermediate step toward problem solving. See concrete examples of how thoughts are generated, evaluated, and searched in Figures 2,4,6.

choices instead of just picking one, and (2) evaluates its current status and actively looks ahead or backtracks to make more global decisions.

To design such a planning process, we return to the origins of artificial intelligence (and cognitive science), drawing inspiration from the planning processes explored by Newell, Shaw, and Simon starting in the 1950s [21, 22]. Newell and colleagues characterized problem solving [21] as search through a combinatorial problem space, represented as a tree. We thus propose the Tree of Thoughts (ToT) framework for general problem solving with language models. As Figure 1 illustrates, while existing methods (detailed below) sample continuous language sequences for problem solving, ToT actively maintains a tree of thoughts, where each *thought* is a coherent language sequence that serves as an intermediate step toward problem solving (Table 1). Such a high-level semantic unit allows the LM to self-evaluate the progress different intermediate thoughts make towards solving the problem through a deliberate reasoning process that is also instantiated in language (Figures 2,4,6). This implementation of search heuristics via LM self-evaluation and deliberation is novel, as previous search heuristics are either programmed or learned. Finally, we combine this language-based capability to generate and evaluate diverse thoughts with search algorithms, such as breadth-first search (BFS) or depth-first search (DFS), which allow systematic exploration of the tree of thoughts with lookahead and backtracking.

Empirically, we propose three new problems that challenge existing LM inference methods even with the state-of-the-art language model, GPT-4 [23]: Game of 24, Creative Writing, and Crosswords (Table 1). These tasks require deductive, mathematical, commonsense, lexical reasoning abilities, and a way to incorporate systematic planning or search. We show ToT obtains superior results on all three tasks by being general and flexible enough to support different levels of thoughts, different ways to generate and evaluate thoughts, and different search algorithms that adapt to the nature of different problems. We also analyze how such choices affect model performances via systematic ablations and discuss future directions to better train and use LMs.

2 Background

We first formalize some existing methods that use large language models for problem-solving, which our approach is inspired by and later compared with. We use p_θ to denote a pre-trained LM with parameters θ , and **lowercase letters** $x, y, z, s, \dots$ **to denote a language sequence**, i.e. $x = (x[1], \dots, x[n])$ where each $x[i]$ is a token, so that $p_\theta(x) = \prod_{i=1}^n p_\theta(x[i]|x[1..i])$. We use uppercase letters $S, \dots$ to denote a collection of language sequences.

Input-output (IO) prompting is the most common way to turn a problem input x into output y with LM: $y \sim p_\theta(y|\text{prompt}_{IO}(x))$, where $\text{prompt}_{IO}(x)$ wraps input x with task instructions and/or few-shot input-output examples. For simplicity, let us denote $p_\theta^{\text{prompt}}(\text{output} | \text{input}) = p_\theta(\text{output} | \text{prompt}(\text{input}))$, so that IO prompting can be formulated as $y \sim p_\theta^{IO}(y|x)$.

Chain-of-thought (CoT) prompting [38] was proposed to address cases where the mapping of input x to output y is non-trivial (e.g. when x is a math question and y is the final numerical answer). The key idea is to introduce a chain of *thoughts* $z_1, \dots, z_n$ to bridge x and y , where each z_i is a coherent language sequence that serves as a meaningful intermediate step toward problem solving (e.g. z_i could be an intermediate equation for math QA). To solve problems with CoT, each thought $z_i \sim p_\theta^{CoT}(z_i | x, z_{1 \dots i-1})$ is sampled sequentially, then the output $y \sim p_\theta^{CoT}(y | x, z_{1 \dots n})$. In practice, $[z_{1 \dots n}, y] \sim p_\theta^{CoT}(z_{1 \dots n}, y | x)$ is sampled as a continuous language sequence, and the **decomposition** of thoughts (e.g. is each z_i a phrase, a sentence, or a paragraph) is left ambiguous.

Self-consistency with CoT (CoT-SC) [36] is an ensemble approach that samples k i.i.d. chains of thought: $[z_{1 \dots n}^{(i)}, y^{(i)}] \sim p_\theta^{CoT}(z_{1 \dots n}, y | x)$ ($i = 1 \dots k$), then returns the most frequent output: $\arg \max_y \#\{i \mid y^{(i)} = y\}$. CoT-SC improves upon CoT, because there are generally different thought processes for the same problem (e.g. different ways to prove the same theorem), and the output decision can be more faithful by exploring a richer set of thoughts. However, within each chain there is no local exploration of different thought steps, and the “most frequent” heuristic only applies when the output space is limited (e.g. multi-choice QA).

3 Tree of Thoughts: Deliberate Problem Solving with LM

A genuine problem-solving process involves the repeated use of available information to initiate exploration, which discloses, in turn, more information until a way to attain the solution is finally discovered.— Newell et al. [21]

Research on human problem-solving suggests that people search through a combinatorial problem-space – a tree where the nodes represent partial solutions, and the branches correspond to operators that modify them [21, 22]. Which branch to take is determined by heuristics that help to navigate the problem-space and guide the problem-solver towards a solution. This perspective highlights two key shortcomings of existing approaches that use LMs to solve general problems: 1) Locally, they do not explore *different* continuations within a thought process – the branches of the tree. 2) Globally, they do not incorporate any type of planning, lookahead, or backtracking to help evaluate these different options – the kind of heuristic-guided search that seems characteristic of human problem-solving.

To address these shortcomings, we introduce *Tree of Thoughts (ToT)*, a paradigm that allows LMs to explore multiple reasoning paths over thoughts (Figure 1(c)). ToT frames any problem as a search over a tree, where each node is a **state** $s = [x, z_{1 \dots i}]$ representing a partial solution with the input and the sequence of thoughts so far. A specific instantiation of ToT involves answering four questions: 1. How to **decompose** the intermediate process into thought steps; 2. How to **generate** potential thoughts from each state; 3. How to heuristically **evaluate** states; 4. What **search** algorithm to use.

1. Thought decomposition. While CoT samples thoughts coherently without explicit decomposition, ToT leverages problem properties to design and decompose intermediate thought steps. As Table 1 shows, depending on different problems, a thought could be a couple of words (Crosswords), a line of equation (Game of 24), or a whole paragraph of writing plan (Creative Writing). In general, a thought should be “small” enough so that LMs can generate promising and diverse samples (e.g. generating a whole book is usually too “big” to be coherent), yet “big” enough so that LMs can evaluate its prospect toward problem solving (e.g. generating one token is usually too “small” to evaluate).

2. Thought generator $G(p_\theta, s, k)$. Given a tree state $s = [x, z_{1 \dots i}]$, we consider two strategies to generate k candidates for the next thought step:

- (a) **Sample** i.i.d. thoughts from a CoT prompt (Creative Writing, Figure 4): $z^{(j)} \sim p_\theta^{CoT}(z_{i+1} | s) = p_\theta^{CoT}(z_{i+1} | x, z_{1 \dots i})$ ($j = 1 \dots k$). This works better when the thought space is rich (e.g. each thought is a paragraph), and i.i.d. samples lead to diversity;
- (b) **Propose** thoughts sequentially using a “propose prompt” (Game of 24, Figure 2; Crosswords, Figure 6): $[z^{(1)}, \dots, z^{(k)}] \sim p_\theta^{propose}(z_{i+1}^{(1 \dots k)} | s)$. This works better when the thought space is more constrained (e.g. each thought is just a word or a line), so proposing different thoughts in the same context avoids duplication.

3. State evaluator $V(p_\theta, S)$. Given a frontier of different states, the state evaluator evaluates the progress they make towards solving the problem, serving as a *heuristic* for the search algorithm to determine which states to keep exploring and in which order. While heuristics are a standard approach to solving search problems, they are typically either programmed (e.g. DeepBlue [3]) or